# DATA STRUCTURES AND ALGORITHMS

Case Study

**Unit V - Graphs and Trees**

GPS Navigation and ZIP/GZIP File Compression


| | |
|---|---|
| **Name** | Rudrakshi Koul |
| **Roll No.** | 09 |
| **College** | Bharati Vidyapeeth's College of Engineering, Lavale |

# Case Study: GPS Navigation and ZIP/GZIP File Compression

Concepts: Weighted Graph, Shortest Path, Huffman Tree

## Part A: GPS / Navigation System (Graph)

- **Model:** intersections or cities are vertices; roads are edges with weights for distance or travel time.
- **Representation:** adjacency list, since road networks are sparse.
- **Shortest route:** Dijkstra's algorithm finds the minimum-cost path from source to destination in O((V + E) log V) with a priority queue.
- **Other uses:** BFS finds the route with fewest stops; a Minimum Spanning Tree (Prim's or Kruskal's) plans cable or pipeline networks at minimum cost.
- **Example:** For Nashik to Pune, the algorithm compares possible road combinations and returns the least-time route.

## Part B: ZIP/GZIP Compression (Huffman Tree)

Idea: give shorter binary codes to frequent characters and longer codes to rare ones.

- Count the frequency of each character.
- Place all characters in a min-priority queue.
- Repeatedly remove the two lowest-frequency nodes and merge them into a parent whose frequency is their sum.
- Continue until one tree remains. Assign 0 to left edges and 1 to right edges to get the codes (the exact bits can differ between valid Huffman trees, but the total length is the same).

**Example:** For the text "AAAABBC":

| Character | Frequency | Code | Bits used |
|---|---|---|---|
| **A** | 4 | 1 | 4 |
| **B** | 2 | 01 | 4 |
| **C** | 1 | 00 | 2 |
| **Total** | 7 | | 10 |

Huffman coding needs **10 bits** instead of 56 bits in 8-bit ASCII. Time complexity is O(n log n).

**Conclusion:** Graphs model real-world networks and enable route optimisation. Trees, especially Huffman trees, enable lossless compression through frequency-based encoding.

## Python Program

```
# Unit V: Dijkstra shortest path (GPS) + Huffman coding (compression)
import heapq
from collections import Counter

def dijkstra(graph, src):
    dist = {v: float("inf") for v in graph}
    dist[src] = 0
    pq = [(0, src)]
```

```
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(pq, (dist[v], v))
    return dist

roads = {  # distances in km (illustrative)
    "Nashik": [("Sinnar", 30), ("Igatpuri", 48)],
    "Sinnar": [("Nashik", 30), ("Sangamner", 45)],
    "Igatpuri": [("Nashik", 48), ("Sangamner", 90)],
    "Sangamner": [("Sinnar", 45), ("Igatpuri", 90), ("Pune", 120)],
    "Pune": [("Sangamner", 120)],
}
print("Shortest distances from Nashik:")
for city, km in dijkstra(roads, "Nashik").items():
    print(f"  {city:10} {km} km")

def huffman_codes(text):
    heap = [[f, i, ch] for i, (ch, f) in enumerate(Counter(text).items())]
    heapq.heapify(heap)
    tick = len(heap)
    while len(heap) > 1:
        a, b = heapq.heappop(heap), heapq.heappop(heap)
        heapq.heappush(heap, [a[0] + b[0], tick, (a, b)]); tick += 1
    codes = {}
    def walk(node, code):
        if isinstance(node[2], str):
            codes[node[2]] = code or "0"
        else:
            walk(node[2][0], code + "0"); walk(node[2][1], code + "1")
    walk(heap[0], "")
    return codes

text = "AAAABBC"
codes = huffman_codes(text)
bits = sum(len(codes[c]) for c in text)
print("Huffman codes:", dict(sorted(codes.items())))
print(f"Compressed: {bits} bits vs {len(text) * 8} bits in ASCII")
```

## Output

```
Shortest distances from Nashik:
  Nashik     0 km
  Sinnar     30 km
  Igatpuri   48 km
  Sangamner  75 km
  Pune       195 km
Huffman codes: {'A': '1', 'B': '01', 'C': '00'}
Compressed: 10 bits vs 56 bits in ASCII
```